

Запуск «Bereke» на чистом компьютере — полная инструкция

Проект: **Django 4.2 + Django REST Framework + PostgreSQL**. Сайт ресторана + REST API + админ-панель.

⚠ САМОЕ ВАЖНОЕ (прочитай до начала)

В git-репозитории **НЕТ**:

Чего нет в репозитории	Почему	Что делать
<code>.venv/</code>	виртуальное окружение	создаётся заново (шаги ниже)
<code>.env</code>	секреты	создать из <code>.env.example</code>
<code>media/</code> (все фото: блюда, галерея, <code>bereke.jpeg</code> , <code>povar.jpeg</code>)	папка загрузок	скопировать вручную со старого компа
Данные БД (21 блюдо, 22 фото галереи, филиалы, брони, пользователь <code>admin</code>)	данные в PostgreSQL, не в коде	перенести дампом БД (раздел 4)
<code>staticfiles/</code>	собирается командой	<code>collectstatic</code>

`schema.sql` в корне — это **только структура таблиц** (без данных). Таблицы создаёт `manage.py migrate`.

Вывод: `git clone` + миграции дадут *пустой* ресторан (без блюд, фото, филиалов). Чтобы был тот же контент — нужно перенести **дамп БД + папку `media/`** (раздел 4).

ВАРИАНТ А — Docker (рекомендую, проще всего)

Ставишь **одну программу** (Docker), она поднимает Python, PostgreSQL и nginx сама.

A1. Установить Docker Desktop

- macOS / Windows: скачать с <https://www.docker.com/products/docker-desktop/>, установить, запустить.
- Проверка: в терминале `docker --version` и `docker compose version`.

А2. Получить код

```
git clone <URL-репозитория> bereke
cd bereke
```

(или скопировать папку проекта целиком — но **без** `.venv/`).

А3. Создать файл окружения `deploy/.env`

```
cp deploy/.env.example deploy/.env
```

Открыть `deploy/.env` и обязательно задать: - `SECRET_KEY` — длинная случайная строка (50+ символов) - `POSTGRES_PASSWORD` — любой надёжный пароль БД - `DEBUG=False` (для прод-стека) или `True` (для теста) - `ALLOWED_HOSTS` — например `localhost,127.0.0.1`

А4. Запустить

```
docker compose --env-file deploy/.env up -d --build
```

Поднимутся 3 контейнера: `db` (PostgreSQL), `web` (Django), `nginx` . Сайт: **`http://localhost/`** (nginx слушает порт 80).

А5. Применить миграции и создать админа (один раз)

```
docker compose exec web python manage.py migrate
docker compose exec web python manage.py createsuperuser
docker compose exec web python manage.py collectstatic --noinput
```

А6. Перенести контент (фото + данные) — см. раздел 4.

Остановить: `docker compose down` (с удалением данных БД: `docker compose down -v`).

ВАРИАНТ В — Вручную (Python + PostgreSQL)

В-macOS

1. Установить Homebrew (менеджер программ)

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/installation)"
```

2. Установить Python 3.12 и PostgreSQL 16

```
brew install python@3.12 postgresql@16
brew services start postgresql@16      # запустить БД как сервис
```

Проверка: `python3.12 --version`, `/opt/homebrew/opt/postgresql@16/bin/pg_isready`

3. Создать базу данных и пользователя

```
PG=/opt/homebrew/opt/postgresql@16/bin
$PG/psql -d postgres -c "CREATE ROLE restaurant LOGIN PASSWORD 'restaurant' CREATEDB;"
$PG/psql -d postgres -c "CREATE DATABASE restaurant OWNER restaurant;"
```

(Имя/пароль должны совпадать с `.env` — по умолчанию всё `restaurant`.)

4. Получить код и окружение

```
git clone <URL-репозитория> bereke && cd bereke
python3.12 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
```

5. Файл `.env`

```
cp .env.example .env
```

Открыть `.env`, задать минимум:

```
SECRET_KEY=<случайная строка 50+ символов>
DEBUG=True
ALLOWED_HOSTS=localhost,127.0.0.1
DB_NAME=restaurant
DB_USER=restaurant
DB_PASSWORD=restaurant
DB_HOST=localhost
DB_PORT=5432
```

6. Таблицы, статика, админ

```
python manage.py migrate
python manage.py collectstatic --noinput
python manage.py createsuperuser      # придумать логин/пароль
```

7. Перенести контент (фото + данные) — раздел 4 ниже.

8. Запуск

```
python manage.py runserver
```

Открыть <http://127.0.0.1:8000/>

B-Windows (кратко)

1. **Python 3.12** — с <https://www.python.org/downloads/> (галочка «Add Python to PATH»).
2. **PostgreSQL 16** — установщик с <https://www.postgresql.org/download/windows/> (запомнить пароль `postgres`).
3. Через **pgAdmin** или `psql` создать БД `restaurant` и пользователя `restaurant`.
4. В PowerShell в папке проекта:

```
powershell python -m venv .venv  
.venv\Scripts\activate pip install -r requirements.txt copy .env.example .env #  
отредактировать .env python manage.py migrate python manage.py collectstatic --  
noinput python manage.py createsuperuser python manage.py runserver
```

4. ПЕРЕНОС КОНТЕНТА со старого компьютера (фото + данные)

Без этого шага ресторан будет **пустой** (нет блюд, галереи, филиалов).

4.1. На СТАРОМ компьютере — сделать дамп БД

```
# путь к pg_dump может отличаться; пример для Postgres 16 (Homebrew):  
/opt/homebrew/opt/postgresql@16/bin/pg_dump \  
-U restaurant -h 127.0.0.1 -d restaurant -Fc -f bereke_db.dump
```

Получится файл `bereke_db.dump`.

4.2. Скопировать на НОВЫЙ компьютер 2 вещи

- файл `bereke_db.dump`
- **всю папку `media/`** (фото блюд, галереи, `bereke.jpeg`, `rovar.jpeg`) → положить её в корень проекта на новом компе.

(Перенос — флешка, облако, `scp` — как удобно.)

4.3. На НОВОМ компьютере — восстановить БД

```
# БД restaurant уже создана (шаг B-3). Восстановить данные:  
/opt/homebrew/opt/postgresql@16/bin/pg_restore \  
-U restaurant -h 127.0.0.1 -d restaurant --clean --if-exists bereke_db.dump
```

При восстановлении из дампа шаги `migrate` / `createsuperuser` не нужны — таблицы, данные и пользователь `admin` уже внутри дампа.

Альтернатива (если переносишь только структуру, без данных): `psql -U restaurant -d restaurant -f schema.sql` затем заполнять через админку.

5. Доступы и проверка

- Сайт: `http://127.0.0.1:8000/` (или `http://localhost/` в Docker)
- Меню: `/menu/` · Бронирование: `/reservation/`
- Админка: `/admin/`
- API: `/api/`

Логин в админку — тот, что задал в `createsuperuser`. (На текущем компе сейчас: `admin` / `admin123` — **смени на проде!**)

Быстрая проверка, что всё поднялось:

```
curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1:8000/      # ждём 200  
curl -s http://127.0.0.1:8000/api/menu/items/ | head -c 200         # должны прийти б
```

6. Траблшутинг (реальные грабли этого проекта)

Симптом	Причина / решение
<code>connection refused</code> к БД	PostgreSQL не запущен → <code>brew services start postgresql@16</code>
Порт 5432 занят	Уже стоит другой PostgreSQL. Либо использовать его, либо сменить <code>DB_PORT</code> .
<code>password authentication failed</code>	Логин/пароль в <code>.env</code> ≠ роли в БД. Пересоздать роль или поправить <code>.env</code> .
Меню/галерея пустые	Не перенесли БД-дамп (раздел 4) — данных нет.
Фото не грузятся (битые картинки)	Не скопировали папку <code>media/</code> (раздел 4.2).
Изменения «не видны» в браузере	Кэш браузера → открыть в режиме инкогнито или Ctrl/Cmd+Shift+R.
<code>NotOpenSSLWarning</code> в логах	Безвредно (особенность Python 3.9 + LibreSSL). На Python 3.12 не будет.
Админка/статика без стилей	Не выполнен <code>python manage.py collectstatic --noinput</code> .

Кратко (шпаргалка, если PostgreSQL и Python уже стоят)

```
git clone <URL> bereke && cd bereke
python3.12 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env                                # отредактировать
python manage.py migrate                             # ИЛИ pg_restore из дампа
python manage.py collectstatic --noinput
python manage.py createsuperuser                     # если не из дампа
python manage.py runserver
# + скопировать папку media/ со старого компа
```